

CAMPUSEATS

Design Benchmark — Services, Contracts & Schema

1. CAPABILITIES

These capabilities represent the distinct jobs or actions performed by the CampusEats system. They are expressed as verbs because they describe what the system is responsible for doing.

NO.	CAPABILITY	DESCRIPTION
1	Manage profile & addresses	The system allows students to manage their profile information and delivery addresses.
2	Browse restaurants & menus	The system allows students to browse available campus restaurants, view their menus, and see food item prices.
3	Add to cart & place order	The system allows students to select food items, add them to a cart, and place an order.
4	Pay online / refund	The system allows students to pay for their orders online and supports refund operations when required.
5	Assign rider & track delivery	The system assigns a rider to an order, handles the delivery process, and allows the student to track the delivery.
6	Send notifications	

NO.	CAPABILITY	DESCRIPTION
		The system sends notifications to students about important order stages such as order placement, payment, delivery, and completion.

2. SERVICE CONTRACTS

Each service exposes a set of operations that other services may call. The contracts specify only the operation name, input, output, and possible errors. Internal implementation details are hidden.

Account Service

Team Member: Mahi Verma

OPERATION	INPUT	OUTPUT	ERRORS
<code>getProfile</code>	<code>userId</code>	Student profile	<code>USER_NOT_FOUND</code>
<code>getAddress</code>	<code>userId</code> , <code>addressId</code>	Delivery address	<code>ADDRESS_NOT_FOUND</code> <code>USER_NOT_FOUND</code>

Catalogue Service

Team Member: Abhishek Jadhao

OPERATION	INPUT	OUTPUT	ERRORS
<code>listRestaurants</code>	<code>campusId</code>	Available restaurants	<code>CAMPUS_NOT_FOUND</code> <code>NO_RESTAURANTS_FOUND</code>
<code>getMenu</code>	<code>restaurantId</code>	Restaurant menu with items and prices	<code>RESTAURANT_NOT_FOUND</code>
<code>checkItem</code>	<code>itemId</code>	Item name, price, availability	<code>ITEM_NOT_FOUND</code>

Orders Service

Team Member: Subham Kumar Mohanty

OPERATION	INPUT	OUTPUT	ERRORS
addToCart	studentId , itemId , quantity	Updated cart status	ITEM_UNAVAILABLE INVALID_QUANTITY
getCart	studentId	Cart items and total amount	CART_NOT_FOUND
placeOrder	studentId , cart contents , addressId , paymentMethod	orderReference, order status, total amount	EMPTY_CART INVALID_ADDRESS ITEM_UNAVAILABLE PAYMENT_FAILED DELIVERY_ASSIGNMENT_FAILED
getOrder	orderReference	Order status and summary	ORDER_NOT_FOUND

Payments & Delivery Service

Team Member: Harsh N Shende

OPERATION	INPUT	OUTPUT	ERRORS
charge	orderReference , amount , paymentMethod	Payment status and confirmation	INVALID_AMOUNT PAYMENT_DECLINED UNSUPPORTED_PAYMENT_METHOD
refund	paymentReference , amount	Refund status	PAYMENT_NOT_FOUND ALREADY_REFUNDED REFUND_FAILED
assignRider	orderReference , delivery address	Assignment status and rider information	NO_RIDER_AVAILABLE ASSIGNMENT_FAILED
trackDelivery	orderReference	Current delivery status	DELIVERY_NOT_FOUND

Notification Service

Team Member: Rishabh Mishra

OPERATION	INPUT	OUTPUT	ERRORS
send	studentId , notification type , message	Notification delivery status	RECIPIENT_NOT_FOUND NOTIFICATION_FAILED
getNotificationStatus	notificationReference	Notification status	NOTIFICATION_NOT_FOUND

3. CENTRAL OPERATION — PLACEORDER

Operation: placeOrder
Service: Orders Service
Owner: Subham Kumar Mohanty

Purpose

The `placeOrder` operation allows a student to place an order by providing the selected items, delivery address, and payment method. It coordinates the required services while hiding the internal implementation from the caller.

Inputs

- `studentId` — identifies the student placing the order.
- `cart` — contains the selected food items and their quantities.
- `addressId` — identifies the delivery address.
- `paymentMethod` — specifies the selected payment method.

Example Request

```
{
  "studentId": 101,
  "cart": [
    {
      "itemId": 25,
      "quantity": 2
    }
  ],
  "addressId": 20,
  "paymentMethod": "UPI"
}
```

Successful Output

- `orderReference` — reference of the newly placed order.
- `status` — current status of the order.
- `totalAmount` — total cost of the order.

Example Response

```
{
  "orderReference": "ORD-1001",
  "status": "PLACED",
  "totalAmount": 240
}
```

Error Cases

- **EMPTY_CART** — the cart contains no items.
- **INVALID_ADDRESS** — the selected delivery address is invalid.
- **ITEM_UNAVAILABLE** — one or more selected items are unavailable.
- **PAYMENT_FAILED** — the payment could not be completed.
- **DELIVERY_ASSIGNMENT_FAILED** — a rider could not be assigned.

Internal Details Hidden from Callers

The caller does not need to know how the order is processed internally. The Orders Service internally:

1. Validates the order request and cart.
2. Calls the Catalogue Service using `checkItem` to verify item availability and item information.
3. Calculates and confirms the order total.
4. Creates and updates the order internally.
5. Calls the Payments & Delivery Service using `charge` to process the payment.
6. Calls the Payments & Delivery Service using `assignRider` to arrange delivery.
7. Calls the Notification Service using `send` to notify the student.
8. Handles failures from these internal service calls and returns an appropriate error to the caller.

4. SERVICE VALIDATION

Each service is checked against the five properties of a service: Reachable, Self-contained, Has a Contract, Independent, and Loosely Coupled.

SERVICE	REACHABLE	SELF-CONTAINED	HAS CONTRACT	INDEPENDENT	LOOSELY COUPLED	FIX
Account Service	PASS	PASS	PASS	PASS	PASS	None
Catalogue Service	PASS	PASS	PASS	PASS	PASS	None
Orders Service	PASS	PASS	PASS	PASS	PASS	None
Payments & Delivery Service	PASS	PASS	PASS	PASS	PASS	None
Notification Service	PASS	PASS	PASS	PASS	PASS	None

Validation Notes

Account Service

- **Reachable:** Other services can access account information through defined operations.
- **Self-contained:** Owns the users and addresses data.
- **Has a Contract:** Provides defined operations for account-related requests.
- **Independent:** Its internal implementation can be changed without requiring other services to access its data directly.
- **Loosely Coupled:** Other services communicate through service operations instead of accessing its data directly.

Catalogue Service

- **Reachable:** Other services can call catalogue operations.
- **Self-contained:** Owns restaurants and menu_items data.
- **Has a Contract:** Provides operations such as checking item availability and retrieving item information.
- **Independent:** Its internal catalogue implementation is hidden from other services.
- **Loosely Coupled:** Other services use catalogue operations instead of accessing catalogue data directly.

Orders Service

- **Reachable:** Other services can access order functionality through defined operations.
- **Self-contained:** Owns carts, cart_items, orders, and order_items data.
- **Has a Contract:** Provides operations such as placeOrder and getOrder.
- **Independent:** Its internal order processing and storage are hidden from callers.
- **Loosely Coupled:** It communicates with other services through their contracts rather than their databases.

Payments & Delivery Service

- **Reachable:** Other services can call payment and delivery operations.
- **Self-contained:** Owns transactions, refunds, riders, and assignments data.
- **Has a Contract:** Provides defined operations for payment and delivery functionality.
- **Independent:** Its internal payment and delivery implementation is hidden from other services.
- **Loosely Coupled:** Other services use payment and delivery operations instead of accessing its data directly.

Notification Service

- **Reachable:** Other services can request notification operations.
- **Self-contained:** Owns the notification data and message log.
- **Has a Contract:** Provides defined notification operations.
- **Independent:** Its internal notification mechanism can be changed without changing other services.
- **Loosely Coupled:** Other services request notifications through its contract instead of accessing its data directly.

Overall Result

All five services satisfy the five required properties of a service. No service currently fails the validation. Therefore, no corrective action is required.

Generated for Assignment 2 — CampusEats System Design Benchmark.